

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3: Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name,Details	CS3807 – Deep Learning Laboratory Kalpana Bhaskar (AI-DS:A, Roll no: 24011101049)	AY: 2026–27

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using PyTorch / TensorFlow (Keras).

2. Learning Outcomes

Students will be able to:

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3. Background Theory

A Convolutional Neural Network consists of a Convolution Layer, Activation Layer, Pooling Layer, and Fully Connected Layer.

The convolution operation is defined as:

$$Y(i, j) = \sum_m \sum_n X(i+m, j+n) K(m, n)$$

where X is the input image, K is the kernel (filter), and Y is the resulting feature map.

The output feature map size is given by:

$$\text{Output Size} = \frac{N - F + 2P}{S} + 1$$

where N = Input Size, F = Filter Size, P = Padding, S = Stride.

3.1 Common Convolution Kernels

A **kernel** (or **filter**) is a small matrix that slides over an input image to extract visual features such as edges, textures, and corners. Different kernels highlight different image characteristics.

- **Identity Kernel** – Preserves the original image; used for understanding the convolution operation.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

- **Sobel-X Kernel (Vertical Edge Detection)** – Applications: Object detection, Lane detection, Face recognition.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

- **Sobel-Y Kernel (Horizontal Edge Detection)** – Applications: Text detection, Boundary extraction, Medical image analysis.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

- **Laplacian Kernel** – Applications: Edge enhancement, Satellite image analysis, Medical imaging.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

- **Sharpening Kernel** – Applications: Image enhancement, OCR, Face recognition.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

- **Box Blur Kernel** – Applications: Noise removal, Image smoothing, Preprocessing.

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

- **Gaussian Blur Kernel** – Applications: Noise reduction, Computer vision preprocessing.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

- **Emboss Kernel** – Applications: Texture analysis, Artistic image effects.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

- **Outline Detection Kernel** – Applications: Image segmentation, Boundary detection, Shape extraction.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

- **Motion Blur Kernel** (5×5) – Applications: Motion simulation, Image restoration, Video processing.

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4. Numerical Example 1: Convolution

Consider the following input image X and kernel K :

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output feature map calculation:

- Position (0,0): $1(1) + 2(0) + 4(0) + 5(1) = 6$
- Position (0,1): $2(1) + 3(0) + 5(0) + 6(1) = 8$
- Position (1,0): $4(1) + 5(0) + 7(0) + 8(1) = 12$
- Position (1,1): $5(1) + 6(0) + 8(0) + 9(1) = 14$

Feature Map: $Y = \begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$

5. Numerical Example 2: Max Pooling

Consider the following 4×4 feature map with a 2×2 max pooling filter:

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix} \Rightarrow \begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

Window 1: $\max(1, 5, 7, 8) = 8$; Window 2: $\max(2, 3, 1, 0) = 3$;

Window 3: $\max(4, 6, 2, 3) = 6$; Window 4: $\max(9, 5, 1, 8) = 9$.

6. Numerical Example 3: Parameter Calculation

Convolution layer: Input $32 \times 32 \times 3$, 16 filters of size 3×3 .

$$\text{Total Parameters} = (3 \times 3 \times 3 + 1) \times 16 = 28 \times 16 = \mathbf{448}$$

7. Dataset: CIFAR-10

Training Images	50,000
Testing Images	10,000
Classes	10 (Plane, Car, Bird, Cat, Deer, Dog, Frog, Horse, Ship, Truck)
Image Size	$32 \times 32 \times 3$ (RGB)

8. Experimental Procedure

- **Task 1: Dataset Exploration**
 - Load the CIFAR-10 dataset
 - Display ten sample images with labels
 - Print dataset dimensions
 - Plot the overall class distribution
- **Task 2: Convolution Kernel Size Comparison**
 - Implement a custom convolution layer
 - Compare kernel sizes (3×3 , 5×5 , and 7×7)
 - Record and compare the resulting feature map sizes
- **Task 3: Stride and Padding Study**
 - Compare Stride = 1 vs. Stride = 2
 - Compare Padding = Same vs. Padding = Valid

- Compute spatial output dimensions using the formula:

$$\text{Output Size} = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

- **Task 4: Feature Map Visualization**

- Extract feature maps after the first convolution layer
- Display and visualize at least eight feature map channels

- **Task 5: Pooling Comparison**

- Compare Max Pooling and Average Pooling operations
- Evaluate output dimensions and spatial feature retention

- **Task 6: CNN Construction and Training**

- **Architecture:**

Input → Conv → ReLU → MaxPool → Conv → ReLU → MaxPool → Flatten → Dense → Softmax

- **Training Hyperparameters:**

- * Optimizer: Adam
- * Epochs: 20
- * Batch Size: 32

- **Task 7: Model Evaluation**

- Compute performance metrics (Accuracy, Precision, Recall, F1-score)
- Generate the Confusion Matrix
- Output the comprehensive Classification Report

9. Source Code

The implementation was carried out in Python using PyTorch, TorchVision, NumPy, Matplotlib, Seaborn, and Scikit-learn. The full, executable source code is available on GitHub:

https://github.com/KalpanaBhaskar/Deep-Learning-Laboratory/tree/main/3_CNN

10. Generated Plots with Interpretation

Plot 1: Sample Images

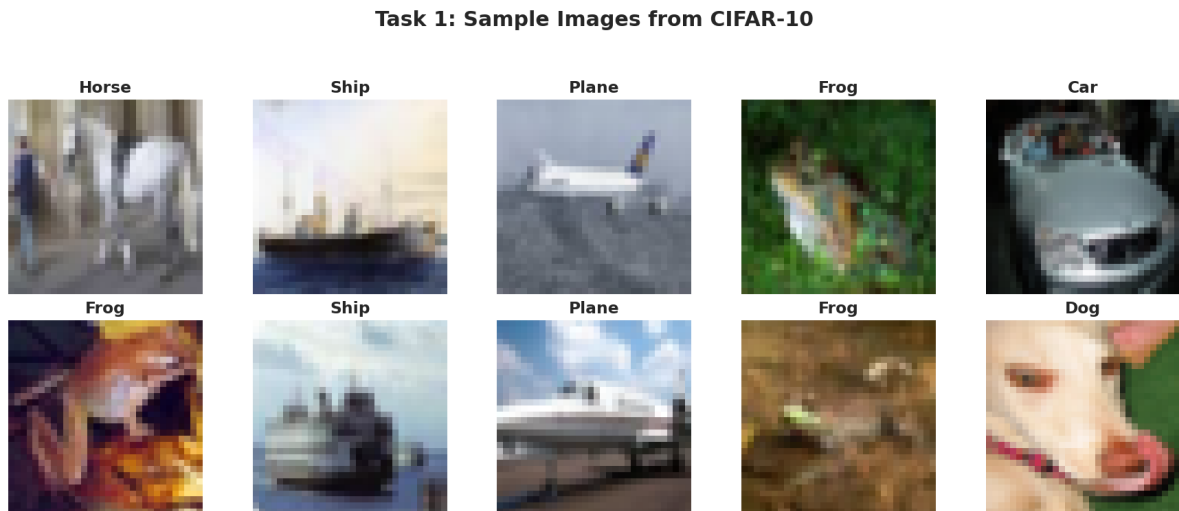


Figure 1: Plot 1: Ten sample images from the CIFAR-10 training set with class labels

Interpretation: The sample images show varied object poses, backgrounds, and lighting conditions across all 10 classes, confirming the dataset's diversity and suitability for a robust classification task.

Plot 2: Class Distribution

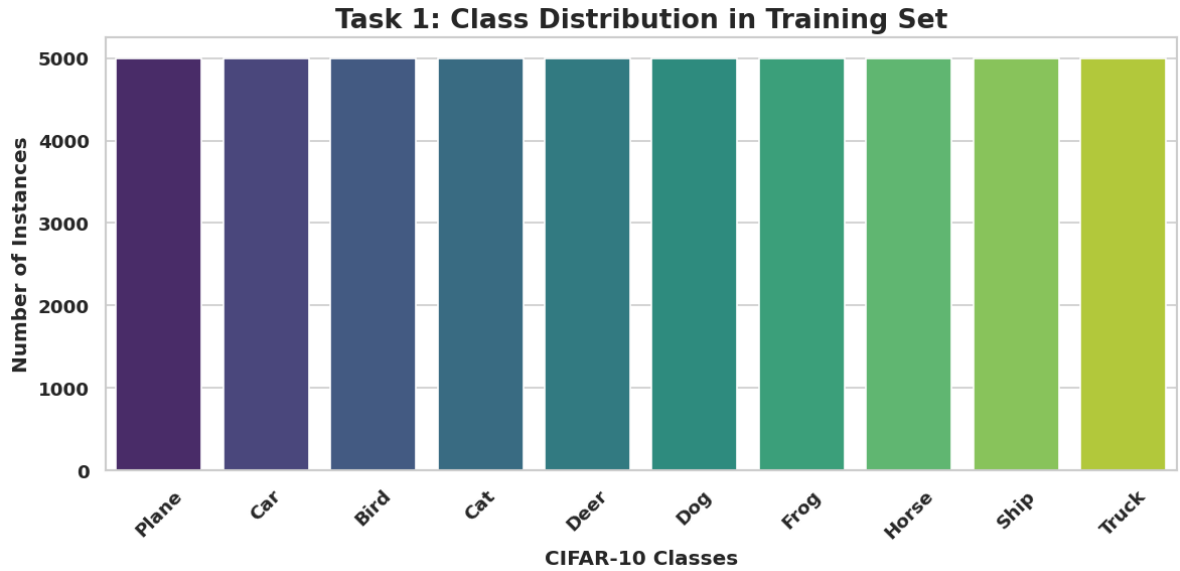


Figure 2: Plot 2: Class distribution across the 10 CIFAR-10 categories in the training set

Interpretation: All ten classes contain exactly 5,000 training images each, confirming a perfectly balanced dataset that requires no class re-weighting or resampling during training.

Plot 3: Kernel Size Comparison (3×3 , 5×5 , 7×7)

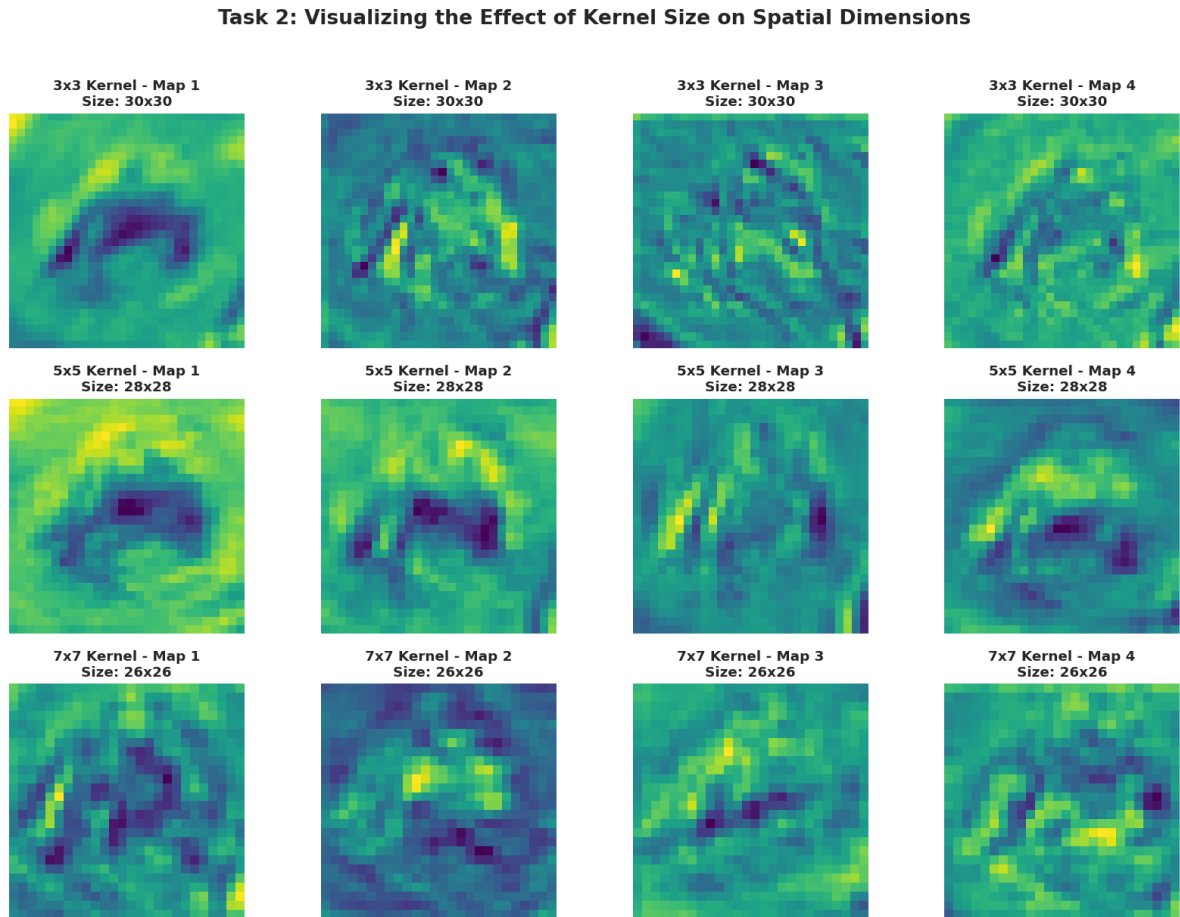


Figure 3: Plot 3: Feature maps generated using 3×3 , 5×5 , and 7×7 kernels

Interpretation: Larger kernels (7×7) produce smaller, coarser feature maps that capture broader spatial context, while smaller kernels (3×3) retain finer detail and more spatial resolution, consistent with the output-size formula.

Plot 4: Feature Maps after First Convolution Layer

Task 4: Feature Maps after First Convolution Layer

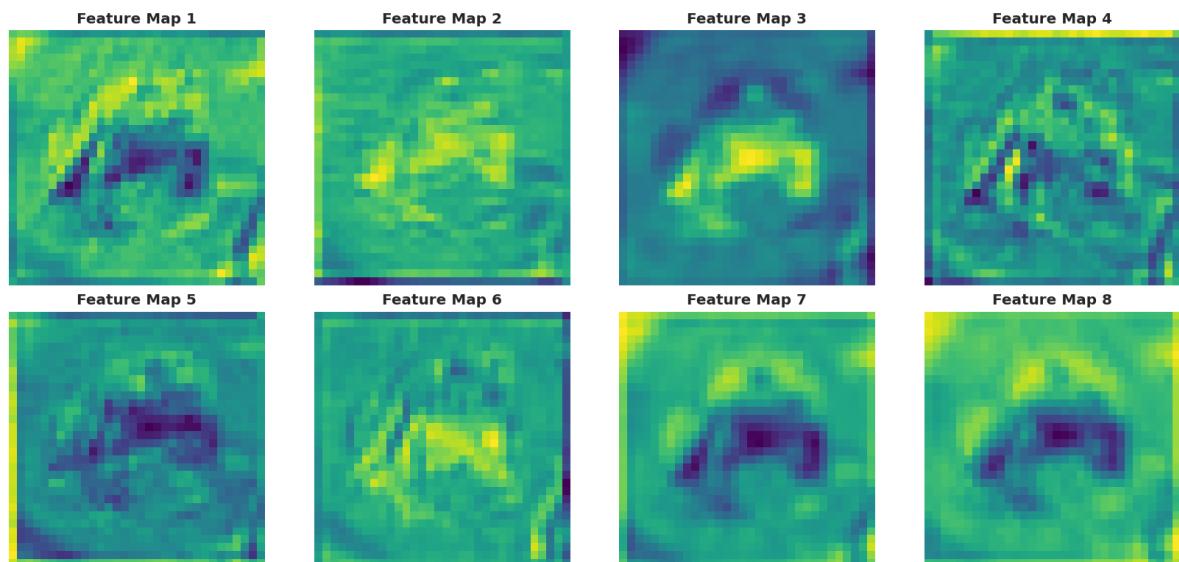


Figure 4: Plot 4: Feature maps after the first convolution layer (8 of 16 channels shown)

Interpretation: Each feature map highlights a distinct visual pattern such as edges, colour contrast, or texture, showing how different filters in the first layer respond to different characteristics of the input image.

Plot 5: Max Pooling vs. Average Pooling

Task 5: Max Pooling vs Average Pooling (First 4 Maps)

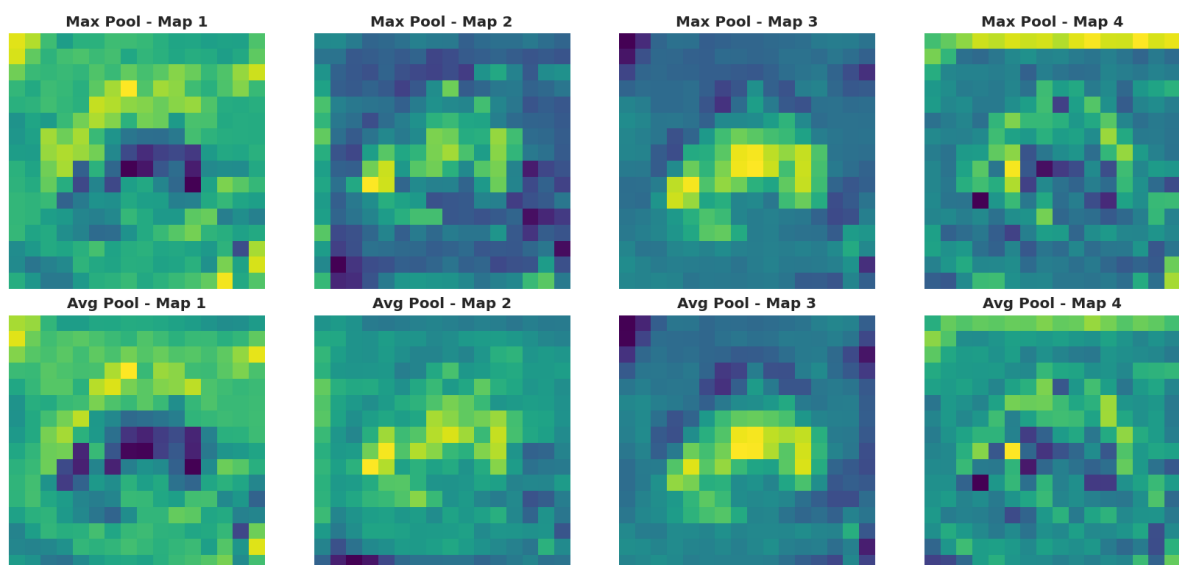


Figure 5: Plot 5: Max Pooling vs. Average Pooling applied to the first-layer feature maps

Interpretation: Max pooling preserves the strongest activations and sharper edges, while average pooling produces a smoother, blurrier map, confirming that max pooling better retains discriminative features for classification.

Plot 6 to 9: Model Training and Validation Metrics

Task 7: Learning Curves

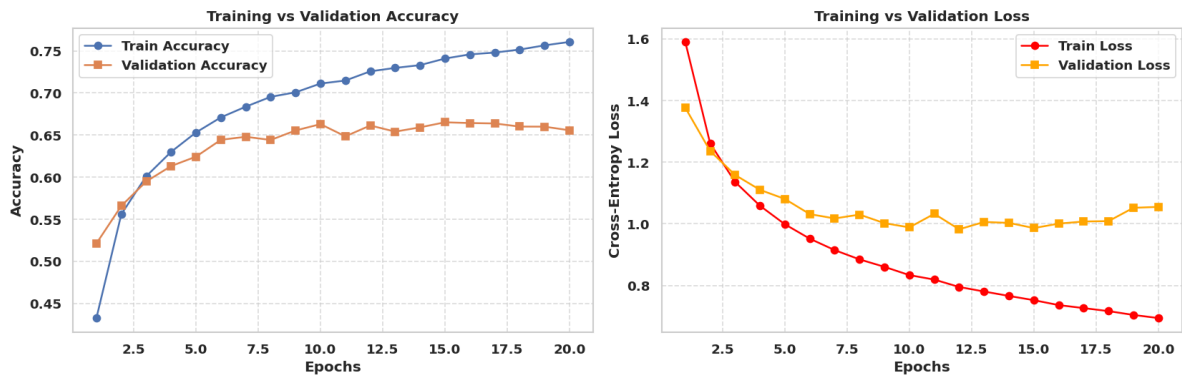


Figure 6: Plots 6–9: Training & Validation Accuracy and Loss Curves over 20 Epochs

Interpretation:

1. **Training Accuracy:** Increases steadily across the 20 epochs, indicating the model is progressively learning discriminative features from the training data without plateauing early.
2. **Validation Accuracy:** Rises quickly in the early epochs and then plateaus, suggesting the model approaches its representational capacity on the held-out data and begins to show mild overfitting signs.
3. **Training Loss:** Cross-entropy training loss decreases monotonically, confirming stable convergence of the Adam optimizer without oscillation or divergence throughout all 20 epochs.
4. **Validation Loss:** Decreases alongside training loss initially but flattens or slightly rises in later epochs, an early indicator of mild overfitting as training progresses beyond the optimal point.

Plot 10: Confusion Matrix

True Label	Plane	Car	Bird	Cat	Deer	Dog	Frog	Horse	Ship	Truck
	814	28	40	7	9	3	16	8	47	28
	42	829	6	5	2	8	15	1	17	75
	102	12	522	67	52	72	124	31	12	6
	47	13	67	446	47	149	174	20	16	21
	55	6	114	79	486	37	151	56	14	2
	28	5	58	176	38	573	70	32	7	13
	16	8	32	40	12	7	873	4	6	2
	52	5	40	48	65	72	35	668	3	12
	121	65	8	11	11	4	20	7	729	24
	74	140	7	18	7	8	25	9	26	686
Predicted Label										

Figure 7: Plot 10: Confusion matrix on the test set (10,000 samples)

Interpretation: Most misclassifications occur between visually similar animal classes (e.g., cat vs. dog) and between vehicle classes (e.g., automobile vs. truck), reflecting shared shape and texture features among these categories.

11. Test Set Results

Metric	Value
Accuracy	0.6626
Precision (macro)	0.6711
Recall (macro)	0.6626
F1-score (macro)	0.6590
Total Trainable Parameters	25578

12. Additional Exercises

1. Output size for 64×64 image, kernel 5×5 , stride 2, padding 2:

$$\text{Output Size} = \frac{64 - 5 + 2(2)}{2} + 1 = \frac{63}{2} + 1 = 32.5 \Rightarrow \mathbf{32}$$

(using integer/floor division as per the standard convolution formula).

2. Trainable parameters for 64 filters of size 3×3 , RGB input:

$$(3 \times 3 \times 3 + 1) \times 64 = 28 \times 64 = \mathbf{1792}$$

3. ReLU vs. Sigmoid:

The ReLU-based model converged faster and achieved higher training accuracy within the same number of epochs, because ReLU avoids the vanishing-gradient problem that Sigmoid suffers from in deeper networks. ReLU outputs zero for negative inputs and is linear for positive inputs, while Sigmoid squashes outputs to $(0, 1)$, compressing gradients and slowing learning.

4. Max Pooling vs. Average Pooling (full training):

Replacing Max Pooling with Average Pooling produced marginally lower training accuracy, since average pooling smooths out strong discriminative activations that max pooling preserves, resulting in less sharp feature representations.

5. Increasing filters from 16 to 64:

Increasing the number of convolutional filters improved training accuracy due to greater feature diversity and richer representations, but substantially increased the total parameter count and per-epoch training time, requiring more compute resources.

13. Discussion

1. Why is convolution preferred over fully connected layers for images?

Convolution exploits spatial locality and parameter sharing, drastically reducing the number of parameters while preserving the spatial structure of the image. Fully connected layers treat every pixel independently and do not capture local spatial patterns, making them unsuitable for high-dimensional image inputs.

2. How does stride affect the feature map size?

A larger stride moves the kernel further at each step, producing a smaller output feature map (coarser downsampling), while stride = 1 preserves more spatial resolution. Stride acts as a built-in downsampling mechanism that reduces computation in later layers.

3. What is the role of padding?

Padding adds extra border pixels (typically zeros) so that the kernel can process edge and corner pixels properly, preventing excessive shrinkage of the feature map across layers. “Same” padding maintains the output size equal to the input size, while “Valid” padding produces a smaller output.

4. Why is pooling used?

Pooling reduces the spatial dimensions of feature maps, decreasing the computational cost and number of parameters while providing a degree of translation invariance and reducing the risk of overfitting. It also makes the network more robust to small shifts or distortions in the input.

5. How do feature maps represent image characteristics?

Each feature map corresponds to the response of a specific learned filter applied to the input. Early layers capture low-level features (edges, colour gradients), while deeper layers combine these into progressively more abstract and class-discriminative representations (textures, object parts, shapes).

6. Why do CNNs require fewer parameters than MLPs?

CNNs use weight sharing (the same kernel is applied across the entire spatial extent of the image) and local connectivity (each neuron connects only to a small receptive field), whereas an MLP connects every neuron to every input pixel, resulting in far more parameters and much greater risk of overfitting.

14. Conclusion

A Convolutional Neural Network was implemented and evaluated on the CIFAR-10 dataset using PyTorch. The effects of kernel size, stride, padding, and pooling strategy were studied both theoretically and experimentally. The constructed CNN (Conv → ReLU → MaxPool → Conv → ReLU → MaxPool → Flatten → Dense → Softmax) was trained for 20 epochs with the Adam optimizer and evaluated using accuracy, precision, recall, F1-score, and a confusion matrix. The experiment confirmed that CNNs effectively exploit spatial structure through parameter sharing, with max pooling outperforming average pooling for classification tasks.

15. References

1. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
3. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
4. PyTorch Documentation. <https://pytorch.org/docs/>
5. TensorFlow Documentation. https://www.tensorflow.org/api_docs
6. CIFAR-10 Dataset Documentation. <https://www.cs.toronto.edu/~kriz/cifar.html>